

Clase 058 — Generación de aleatoriedad segura (CSPRNG)

Parte: 2 — **Criptografía aplicada** · Fuente: *Serious Cryptography* (Aumasson) y NIST SP 800-90A 

Duración estimada: 90 min · Nivel: **Intermedio**

Objetivo

Comprender por qué la aleatoriedad es el cimiento silencioso de toda la criptografía: claves, nonces, IVs, salts y tokens dependen de ella. El alumno aprenderá la diferencia entre un PRNG estadístico (predecible) y un CSPRNG (criptográficamente seguro), de dónde viene la entropía del sistema operativo, y por qué fallos de aleatoriedad han roto sistemas reales (Debian OpenSSL 2008, claves RSA con primos compartidos).

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Distinguir** PRNG estadístico de CSPRNG y sus garantías.
2. **Identificar** las fuentes de entropía del sistema operativo.
3. **Generar** claves, nonces y tokens con APIs seguras (`secrets` , `os.urandom`).
4. **Reconocer** los fallos históricos por mala aleatoriedad.
5. **Evitar** anti-patronos como sembrar con el tiempo o usar `random` para seguridad.

Temas

#	Tema	Por qué importa
1	Entropía y fuentes	Origen de la aleatoriedad
2	PRNG vs CSPRNG	Predecible vs seguro
3	<code>/dev/urandom</code> y <code>getrandom()</code>	API del SO
4	DRBG (SP 800-90A)	Generadores deterministas seguros
5	APIs seguras (<code>secrets</code>)	Uso correcto en código
6	Fallos famosos	Aprender de desastres
7	Pruebas de aleatoriedad	Detectar sesgos

Definiciones y características

- **Entropía**: medida de imprevisibilidad. El SO la recolecta de eventos físicos (interrupciones, ruido de hardware).
- **PRNG (estadístico)**: genera secuencias que "parecen" aleatorias pero son predecibles si conoces el estado/semilla (p. ej. Mersenne Twister). **No** para seguridad.

- **CSPRNG**: PRNG que, aun conociendo salidas previas, no permite predecir las siguientes ni recuperar el estado. Base de claves y nonces.
- **DRBG**: generador determinista por bits recomendado por NIST (Hash_DRBG, HMAC_DRBG, CTR_DRBG); se re-siembra con entropía.
- **getrandom() / /dev/urandom**: interfaz del kernel que entrega bytes de un CSPRNG bien sembrado.
- **Semilla (seed)**: valor inicial; si es predecible (p. ej. el tiempo), toda la salida lo es.
- **Sesgo**: desviación de la uniformidad; se detecta con pruebas estadísticas (Dieharder, NIST STS).

Herramientas y preparación

```
python3 -c "import secrets; print('ok')"  
# opcional: pruebas estadísticas  
which rngtest dieharder 2>/dev/null || echo "opcional"
```

Todo el trabajo es local y de análisis.

Laboratorio guiado

1. **Genera material aleatorio seguro** en Python:

```
python import os, secrets clave = os.urandom(32) # 256 bits para AES nonce = os.urandom(12) #  
nonce GCM token = secrets.token_urlsafe(32) # token de sesión print(clave.hex(), token)
```

2. **Contraejemplo predecible**. Muestra que `random.seed(tiempo)` + `random.getrandbits` produce salidas reproducibles si el atacante conoce el instante; concluye que `random` **no** sirve para seguridad.
3. **Analiza sesgos**. Genera 1 MB con un CSPRNG y con un PRNG mal sembrado; corre pruebas estadísticas (o cuenta frecuencias de bits) y compara.
4. **Estudia el fallo Debian 2008**. Investiga cómo un parche que redujo la entropía dejó el espacio de claves en apenas ~32.767 posibilidades y por qué hubo que regenerar millones de claves.
5. **Nonces únicos**. Simula la generación de nonces para AES-GCM y verifica (con un conjunto) que no se repiten en el volumen esperado.

Ejercicios

1. Explica la diferencia entre `random` y `secrets` en Python.
2. Genera un token de recuperación de 256 bits y razona su espacio.
3. Investiga el incidente de claves RSA con primos compartidos (factorización por MCD).
4. Describe cómo el kernel recolecta entropía en el arranque.
5. Demuestra que sembrar con `time()` produce salidas predecibles.
6. Corre una prueba estadística sobre dos fuentes y compara resultados.

Reto verificable

Escribe una utilidad que genere claves, nonces y tokens exclusivamente con un CSPRNG, y un test que verifique ausencia de repeticiones en una gran muestra de nonces y una distribución de bits cercana a

50/50. **Criterio de aceptación:** la muestra no presenta nonces repetidos y la proporción de unos/ceros se desvía menos de un umbral razonable; el código no usa ningún PRNG estadístico.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Claves reproducibles	Semilla predecible; usa <code>os.urandom</code> / <code>secrets</code>
Uso de <code>random</code> / <code>rand()</code> para claves	No es CSPRNG; sustitúyelo
Nonces repetidos	Generación insegura; usa CSPRNG o contador único
Baja entropía en arranque	Bloquea hasta sembrar (<code>getrandom</code>) o añade fuente de hardware
Reutilizar tokens de sesión	Genera uno nuevo por sesión con suficiente longitud

? Preguntas frecuentes

? **¿/dev/urandom o /dev/random?** En Linux moderno, `urandom` / `getrandom()` es la elección correcta una vez sembrado; `random` puede bloquear innecesariamente.

? **¿Cuántos bits necesito para un token?** Al menos 128 bits (16 bytes) de aleatoriedad real; 256 bits para márgenes amplios.

? **¿Por qué la mala aleatoriedad es tan peligrosa?** Porque compromete claves y nonces de golpe; un CSPRNG débil hace inútil el mejor algoritmo.

🔗 Referencias

- NIST SP 800-90A Rev.1 (DRBG) — <https://csrc.nist.gov/publications/detail/sp/800-90a/rev-1/final>
- Aumasson, *Serious Cryptography*, cap. 2.
- Heninger et al., "Mining Your Ps and Qs" — <https://factorable.net/>
- Debian OpenSSL predictable PRNG (CVE-2008-0166).

➡ Siguiente clase

Clase 059 - Cifrado autenticado (AEAD)